COS341 2026 - Semester Practical, Phases 1, 2a and 2b: User Manual - Group 28

Group Members:

- Brayden Butler - 24824713
- Nathan Chisadza - 24825532
- Finnley Wyllie - 24754120
- Daniel Cohen - 24772756

---

1. What This Program Does

This program is the front-end and semantic analyser of a compiler for the Students' Programming Language (SPL), as defined in the COS341 2026 specifications. Given a plain ASCII text file containing an SPL program, it runs the following phases in order and stops at the first phase that finds an error:

1. **Lexing and parsing (Phase 1):** a lexer and an SLR(1) parser check the syntax and write the syntax tree to `tree.xml`.
2. **Name and scope analysis (Phase 2a):** checks that every variable and function is declared where it is used, that nothing is declared twice at the same scope level, that local variables do not mask a function's parameters, that calls are strictly hierarchic (no recursion), and gives every entity a unique internal name (`sys1`, `sys2`, ...) in the symbol table.
3. **Type analysis (Phase 2b):** gives every node of the syntax tree a type (`unknown`, `ok`, `numeric`, `boolean` or `procedure`) and checks the type rules, for example that a void function is only called as an instruction and a num function only inside a term. It also applies the float/integer rule: a program may not use both `mod` and `div`, and a program that uses `mod` may not contain decimal numbers.

2. How to Run the Program

The submitted executable is named:

```
group-28.jar
```

To run it, open a command prompt / terminal in the folder containing the executable, and run:

```
java -jar group-28.jar SPL.txt tree.xml
```

Where:

- `SPL.txt` is the path to the plain-text SPL source file to be analysed.

- `tree.xml` is the path where the syntax tree is written if parsing succeeds (optional, the default is `tree.xml`).

Optional flags, added after the file names, print the results of the semantic phases for inspection:

- `--dump-symbols` prints the symbol table with the unique internal names.
- `--dump-types` prints the type of every symbol and tree node, and whether the program works with integers (`int`) or floats (`float`).

Example:

```
java -jar group-28.jar examples/SPL.txt tree.xml --dump-symbols --dump-types
```

3. Expected Behaviour

On SUCCESS (a syntactically and semantically correct SPL program):

- The program prints a short confirmation message to the console.
- A file is written at the given output path (e.g. `tree.xml`) containing the syntax tree, with:
  - a root node for the `SPL_PROG` start symbol,
  - inner nodes for every non-terminal in the derivation,
  - leaf nodes for every terminal token consumed from the input,
  - each with a unique ID, its contents, and (for non-root nodes) a reference to its parent's ID; and (for root/inner nodes) a list of its children's IDs.
- No error messages are printed by the name/scope and type analysis.
- The program exits with status code 0.

On FAILURE:

- A lexical or syntax error stops the program before `tree.xml` is written.
- A name/scope or type error is reported after `tree.xml` has been written (the syntax itself was correct).
- Every error is printed to the console (stderr) with the line and column where it was detected, and an explanation or hint, in one of these forms:
  - `Lexical error at line L, column C: ...`
  - `Syntax error at line L, column C: ...`
  - `Semantic error at line L, column C: ...` (Phase 2a, names and scopes)
  - `Type error at line L, column C: ...` (Phase 2b, types)
- All name/scope errors (or all type errors) of a program are listed, not only the first one.
- The program exits with a non-zero status code.

Examples of type errors:

```
Type error at line 3, column 6: A float-integer-conflict might perhaps be possible. ...
Type error at line 3, column 6: function '#hello' is a void function (type procedure), so it
returns no value, but it is used as a term, ...
```

4. Notes on the SPL Language

- Every token in an SPL program must be terminated by a blank space (space, or newline/return).
- User-defined names (variables and function names) must begin with `#`, e.g. `#x`, `#counter`, `#add1`.
- The end-of-file marker `$` referenced in the grammar is a theoretical symbol only. It must never be typed in an input file, and it will not appear in the generated `tree.xml` output.
- All variables are numeric. Functions declared with `void` are procedures and can only be called as an instruction; functions declared with `num` return a number and can only be called inside a term.

5. Tests

The folders `tests/phase2a` and `tests/phase2b` contain small SPL programs: files starting with `v` must be accepted, files starting with `e` must be rejected with the stated number of errors. After compiling the sources to `target/classes`, run them from the repository root with:

```
sh tests/phase2a/run_tests.sh
sh tests/phase2b/run_tests.sh
```

6. System Requirements

- Java 17 or newer.
- No other installation required; the executable is self-contained and can be run directly.